

Una fórmula barata para el β óptimo de una crossbar memristiva

Síntesis del proyecto: de la idea original a un reemplazo validado del grid-search

Walter Quiñonez

Trabajo realizado con Claude Code · Proyecto final del curso

*El softmax de una crossbar memristiva es $\text{softmax}(\beta \cdot I)$, y elegir β por grid-search+K-fold es caro. Este proyecto encuentra una fórmula de dos factores (σ_W , dispersión de pesos; paso_medio, granularidad de la curva potenciación/depresión) que predice el β óptimo **antes de entrenar**, la valida contra 30 grid-searches reales completos ($R^2 = 0.930$), y la usa para reemplazar la grilla ciega de `search_best_beta()` por una búsqueda angosta centrada en la predicción —bajando el regret medido de 10.19 puntos porcentuales a 0.08.*

Índice

1. El problema	2
2. Punto de partida teórico	2
3. Idea original y sus sesgos	2
4. Primera validación (6 curvas reales)	3
5. La fórmula combinada	3
6. Reconfirmación a escala (18 $\rightarrow$ 30 curvas)	3
7. Payoff práctico: una búsqueda mejorada	4
8. Integración a producción	5
9. Chequeos de robustez adicionales	5
10. Síntesis cuantitativa del hilo completo	6
11. Limitaciones honestas	6
12. Conclusión y recomendación	7
13. Registro de procedencia	7

1. El problema

En una red neuronal implementada sobre una crossbar memristiva, cada corriente de salida $I_i = \sum_j (G_{ij}^+ - G_{ij}^-) V_j$ pasa por una no linealidad $O_i = f(\beta \cdot I_i)$ (softmax o tanh, según la capa). El valor de β determina si esa no linealidad satura o queda demasiado chata, y el accuracy final depende fuertemente de él. El único método disponible en el código base para elegirlo, `search_best_beta()`, es un grid-search con K-fold: cientos de entrenamientos cortos por cada β candidato. La pregunta de este proyecto: ¿se puede estimar un β cercano al óptimo sin entrenar nada, usando solo propiedades de la curva de potenciación/depresión (P/D) del dispositivo memristivo?

2. Punto de partida teórico

`papers/Reporte_beta_optimo_analitico.pdf` (reporte previo del proyecto) deriva una fórmula candidata modelando la corriente como una suma de N términos aproximadamente independientes (Teorema Central del Límite), siguiendo el mismo esquema $I = \sum_j W_{ij} V_j$, $f = \tanh(\beta I)$ que Prezioso et al. (*Nature* 521, 61, 2015) atribuyen a la recomendación de LeCun et al. (*Efficient Backprop*, 1998):

$$\sigma_I = \sqrt{N} \cdot \sigma_W \cdot V_{rms} \quad \Rightarrow \quad \beta_{\text{optimo}} \approx \frac{\kappa}{\sqrt{N} \sigma_W V_{rms}}$$

con σ_W el desvío estándar de los pesos $W = G^+ - G^-$ bajo la inicialización aleatoria del código, $V_{rms} = \sqrt{E[V^2]}$ de los voltajes de entrada (píxeles de MNIST), y κ una constante $O(1-10)$.

3. Idea original y sus sesgos

`analisis_beta_optimo.py` (raíz del repo) fue el primer intento de poner a prueba esta idea con un Monte Carlo de pesos y corrientes. `analisis_corrientes_beta_optimo/` encontró que ese script tenía dos sesgos que rompían la premisa $E[I] \approx 0$ que asume la fórmula: armaba los pesos como una matriz combinatoria $pot_i - dep_j$ en vez de sortear G^+ y G^- independientemente (como hace la red real vía `G0_initialization`), y muestreaba los voltajes de los valores únicos de píxel en vez de respetar que el 81 % de los píxeles reales de MNIST son exactamente 0.

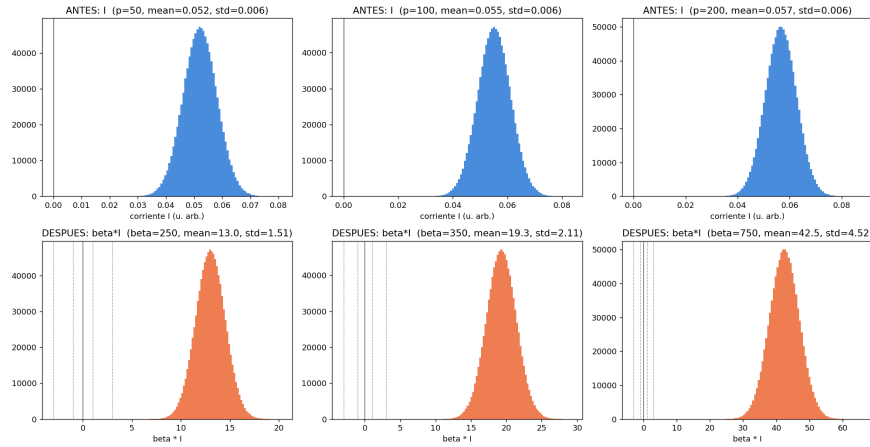


Figura 1: Distribución de corriente Monte Carlo antes y después de corregir los dos sesgos identificados.

4. Primera validación (6 curvas reales)

validacion_formula_beta_optimo/ puso la fórmula ($\kappa = 1$) a prueba contra **6 grid-searches reales completos** (100 épocas, 20 series, 5 folds cada uno) —no una búsqueda reducida. Resultado: **acierta el orden de magnitud** (κ implicado ≈ 0.97 , dentro de $O(1-10)$) y **la dirección entre ventanas de conductancia distintas** (σ_W más grande $\Rightarrow \beta$ óptimo más chico), pero **no explica** la dependencia con el número de pulsos p de la curva P/D a INL nominal fijo: β óptimo sube claramente con p mientras σ_W se mantiene casi constante por diseño del experimento.

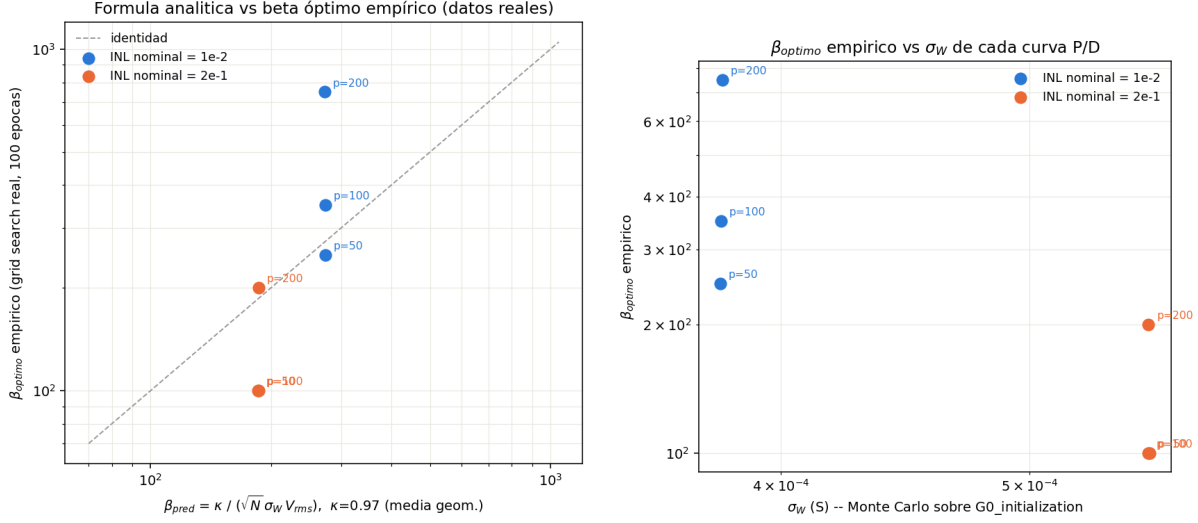


Figura 2: Izq.: β predicho ($\kappa = 1$) vs. empírico. Der.: β óptimo vs. σ_W —acierta la dirección entre grupos de INL, no dentro de un grupo.

5. La fórmula combinada

formula_beta_optimo_propuesta/ combina σ_W (predice la dirección entre ventanas de conductancia) con el **paso medio** de la curva de potenciación, $\text{mean}(|\text{pot}_{i+1} - \text{pot}_i|)$ (predice la dirección con p), usando el Monte Carlo ya corregido:

$$\beta_{\text{optimo}} \approx 1.12 \times 10^{-11} \cdot \sigma_W^{-3.02} \cdot \text{paso_medio}^{-0.64}$$

Modelo	R^2
solo σ_W	0.678
solo paso_medio	0.273
combinado	0.947

Cuadro 1: Regresión log-log sobre los mismos 6 puntos de la Sección 4.

6. Reconfirmación a escala ($18 \rightarrow 30$ curvas)

reconstruccion_beta_optimo_18curvas/ y luego reconstruccion_beta_optimo_30curvas/ repiten el mismo método (sin reentrenar nada —Monte Carlo barato + reconstrucción determinista de

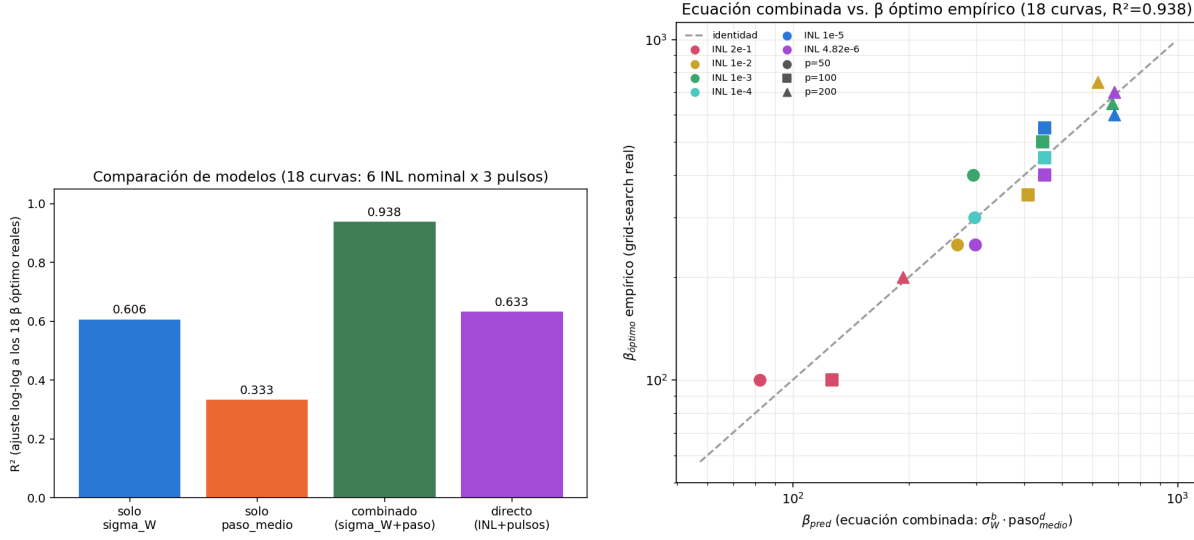


Figura 3: Izq.: R^2 de cada modelo. Der.: β predicho por la ecuación combinada vs. empírico.

logits ya guardados) sobre 18 y luego 30 curvas P/D reales (10 valores de INL nominal $\times$ 3 recuentos de pulsos). Los coeficientes se mantienen prácticamente idénticos al triplicar-y-medio la muestra:

	6 curvas	18 curvas	30 curvas
exponente de σ_I	−3.02	−3.06	−3.00
exponente de paso_medio	−0.64	−0.61	−0.62
R^2 (log-log)	0.947	0.938	0.930

Más importante que el R^2 : para cada una de las 30 curvas se buscó, en la **grilla real de 40 β** ya corrida, el punto más cercano a la predicción de la fórmula, y se comparó su accuracy contra el pico real de esa curva completa. **Las 30 de 30 curvas alcanzan ≥ 99 % de su accuracy pico** (media 99.95 %, mínimo 99.38 %) usando la β predicha, sin ninguna búsqueda —a pesar de que el error puntual de la predicción llega hasta ± 32 % en algunas curvas, porque el pico accuracy-vs- β es ancho (meseta) cerca del óptimo.

7. Payoff práctico: una búsqueda mejorada

`mejora_search_best_beta/` usa este resultado para reemplazar la grilla ciega de `search_best_beta()` (en producción: solo 2 puntos, $\beta \in \{1, 300\}$) por una grilla log-espaciada angosta, centrada en la predicción de la fórmula, con expansión automática si el óptimo cae en el borde. Validación empírica (10 corridas por método, 2 combos representativos, midiendo *regret* real contra el ground truth de 100 épocas ya guardado):

Método	regret medio	regret mediana	regret peor caso	tiempo
(a) original, como se usa hoy	10.19 pp	10.19 pp	20.30 pp	11.8 s
(b) original, defaults de fábrica	0.24 pp	0.03 pp	2.16 pp	118.2 s
(c) mejorado	0.08 pp	0.04 pp	0.45 pp	218.2 s

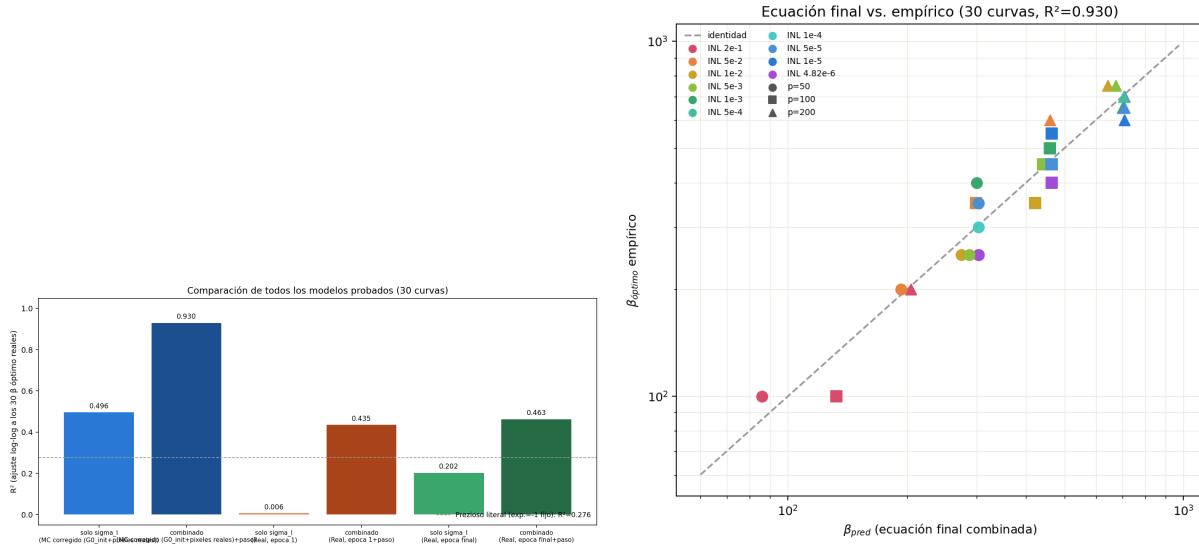


Figura 4: Izq.: la forma literal de Prezioso/LeCun (exponente fijo en -1) da $R^2 = 0.276$; dejar el exponente libre + paso_medio da $R^2 = 0.930$. Der.: β predicho vs. empírico, 30 curvas.

El método (a) —el que corre hoy en `Crossbar_train_experimento_beta_por_capa.py`— elige $\beta = 1$ en el combo más no lineal probado, perdiendo ~ 20 puntos porcentuales de accuracy frente al óptimo real. Validado además contra los 30 combos completos (no solo los 2 de la tabla): la fórmula predice el β óptimo real dentro de un factor $1.26\times$ en el peor de los 30 casos (mediana del cociente predicho/real: 0.948 ; 100 % de los casos dentro de un factor $2\times$).

8. Integración a producción

Los pasos 3–7 documentan la fórmula validándose contra evidencia real, siempre como código nuevo que importa la clase base sin tocarla (ver Sección 13). Una vez validada la mejora de la Sección 7 como un parche aparte (subclase `MemDNNMejorado` en `mejora_search_best_beta/search_best_beta_mejorado.py`), se integró directamente en la clase base `MemDNN` (`MemCrossbarClass_beta_por_capa.py`): un método nuevo `search_best_beta_mejorado()` con las mismas fórmulas y correcciones (common random numbers, grilla log-espaciada centrada en `beta_pred_formula()`, meseta 1-SE), y `Crossbar_train_experimento_beta_por_capa.py` ya lo usa por defecto cuando `optimize_beta=True`, en vez del `search_best_beta()` original (que queda en el archivo, sin uso por defecto). Es la **única** modificación al código base de todo este hilo, y es posterior —y consecuencia directa— de la validación de las Secciones 4 a 7, no una premisa de ellas.

9. Chequeos de robustez adicionales

Tres estudios adicionales pusieron a prueba los bordes del resultado anterior: `validacion_rango_conductancia/` (sensibilidad al ratio G_{max}/G_{min}), `mejora_estimacion_beta_INL_2e1_a_1e3/` (reestimación en la zona de transición de INL) y `reconstruccion_logits_val_INL_1e-2_p_100_beta1/` (sanity check: los logits reconstruidos por Monte Carlo coinciden con los logits reales guardados del entrenamiento). Ver el reporte de cada

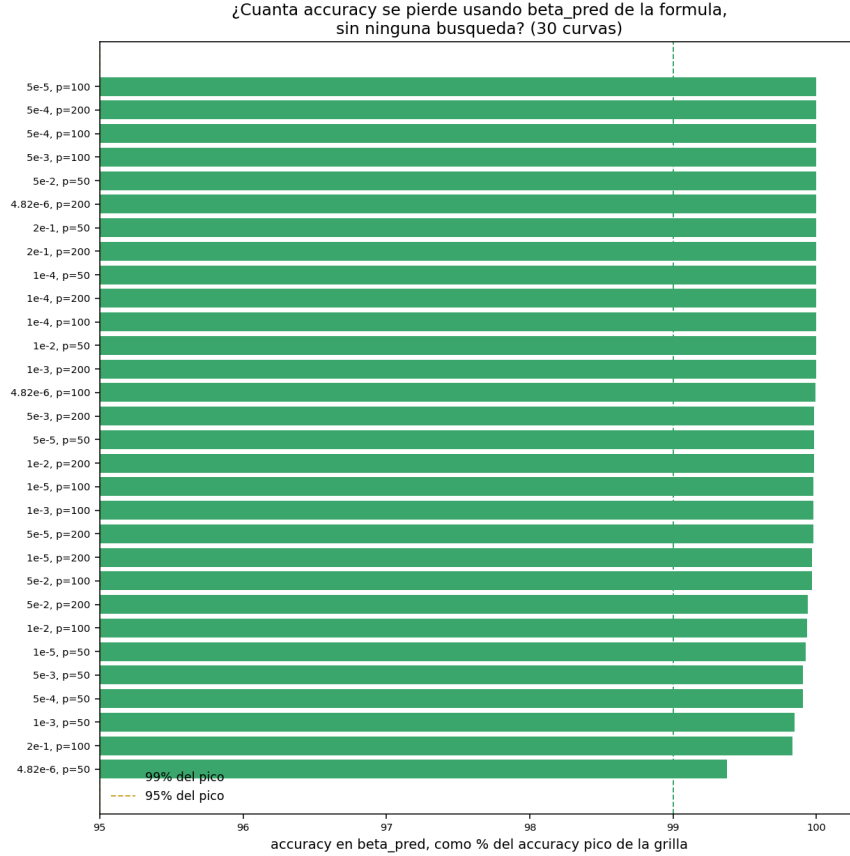


Figura 5: β predicho contra el punto de la grilla real de 40 β más cercano, para las 30 curvas.

carpeta para el detalle.

10. Síntesis cuantitativa del hilo completo

Paso	n curvas	Modelo	R^2 (log-log)
Validación fórmula analítica (solo σ_W)	6	$\beta = \kappa/\sigma_W$	0.678
Fórmula propuesta (σ_W + paso medio)	6	2 factores	0.947
Reconstrucción, escala $3\times$	18	2 factores	0.938
Reconstrucción, escala $5\times$	30	2 factores	0.930

Cuadro 2: Los coeficientes del modelo de 2 factores casi no cambian entre 6, 18 y 30 curvas —la señal más fuerte de que no es un sobreajuste a la muestra chica inicial.

11. Limitaciones honestas

- **N fijo.** Las 30 curvas comparten $N = 784$ (fan-in de MNIST); el exponente teórico de N ($N^{-1/2}$) no se vuelve a poner a prueba con esta data.
- **Un solo dataset.** Fashion-MNIST está disponible pero no se usó en este hilo.

- **Ajuste in-sample.** Los R^2 se calculan sobre las mismas curvas usadas para ajustar los coeficientes; la estabilidad de los coeficientes al escalar $6 \rightarrow 18 \rightarrow 30$ es la evidencia disponible en su lugar, no un reemplazo formal de un held-out set.
- **Grilla de β de 40 puntos, espaciados de a 50.** El β óptimo empírico contra el que se ajusta y valida tiene ese error de resolución incorporado.
- **Solo β global.** Ni la fórmula ni la búsqueda mejorada optimizan β distinto por capa (hilo de trabajo separado, `exploracion_beta_capa_oculta_100/`, fuera del alcance de esta entrega).
- **Errores puntuales de hasta ± 32 %** se traducen en pérdida de accuracy despreciable (≤ 1 %) porque el pico accuracy-vs- β es ancho, pero eso no es lo mismo que decir que la fórmula es precisa en β —es precisa en el resultado práctico que importa.

12. Conclusión y recomendación

La fórmula $\beta_{\text{óptimo}} \approx C \cdot \sigma_I^{-3} \cdot \text{paso_medio}^{-0.6}$ (constante y exponentes según la Sección 6) predice, sin entrenar nada, un β que alcanza ≥ 99 % del accuracy pico en las 30 configuraciones reales disponibles, y usada para centrar `search_best_beta()` baja el regret medio de 10.19pp a 0.08pp con un costo de cómputo despreciable frente a un entrenamiento completo. **Recomendación:** usarla como pre-filtro barato antes de una búsqueda angosta de verificación —no como reemplazo de la búsqueda en un solo paso sin verificar, dado que sigue siendo evidencia sobre un único régimen (arquitectura SP, $N = 784$, MNIST, ratio $G_{\max}/G_{\min} = 10$).

13. Registro de procedencia

Cada carpeta del hilo (Secciones 3–7 y 9) tiene, en su propio reporte, una tabla “registro de códigos usados” con: qué script es nuevo, qué función se importó sin modificar y de qué archivo, y qué dato de entrada se usó. Este reporte enlaza esas tablas en vez de repetirlas. El código base (`MemCrossbarClass_beta_por_capa.py`, `Crossbar_train_experimento_beta_por_capa.py`) no fue modificado por ninguno de esos reportes de validación —la única modificación es la integración a producción descrita en la Sección 8, posterior a toda la validación. Mapa completo del repositorio, entorno y cómo reproducir cada paso: ver `README.md` en la raíz del repositorio.

Generado a partir de reportes ya existentes y verificados de este mismo proyecto; no se corrió ningún entrenamiento ni Monte Carlo nuevo para armar esta síntesis.